



Universidade do Minho
Escola de Engenharia

Relatório 2ª Fase

Laboratórios de Informática III

Grupo 13 | 2025/2026

Bruno Magalhães (a110274)

Diogo Azevedo (a109727)

Vera Almeida (a110723)

Índice

1. Introdução -----	3
2. Desenvolvimento -----	4
2.1. Ajustes realizados -----	4
2.1.1. Indicações dos Docentes -----	4
2.2. Arquitetura Aplicada-----	5
2.3. <i>Queries</i> -----	6
2.3.1. <i>Query 1</i> -----	6
2.3.2. <i>Query 2</i> -----	7
2.3.3. <i>Query 3</i> -----	7
2.3.4. <i>Query 4</i> -----	8
2.3.5. <i>Query 5</i> -----	8
2.3.6. <i>Query 6</i> -----	8
2.4. Programa Interativo -----	9
2.5. Dificuldades sentidas -----	9
3. Conclusão-----	10

1. Introdução

O presente relatório apresentará informações relativas à 2ª Fase do Projeto da Unidade Curricular Laboratórios de Informática III, pertencente ao 2º Ano da Licenciatura em Engenharia Informática, realizada no ano letivo 2025/26, na Universidade do Minho.

O projeto consiste na implementação de uma base de dados em memória, recorrendo a ficheiros no formato .csv fornecidos pela equipa docente, os quais contêm dados relativos a viagens, nomeadamente voos, reservas e passageiros. Esta base de dados tem como objetivo responder a um conjunto de *queries*, explorando conceitos de organização e gestão eficiente da informação, sendo capaz de processar e analisar grandes volumes de dados relativos a informações aéreas.

O sistema, responsável pela gestão de entidades sofreu uma evolução substancial entre a primeira e a segunda fase do projeto. Na Fase 2, foram introduzidos requisitos de desempenho significativamente mais exigentes, decorrentes do aumento substancial do volume de dados e da ampliação do conjunto de consultas a satisfazer. Esta alteração estrutural exigiu uma reavaliação das estratégias de organização e armazenamento da informação. Adicionalmente, foi desenvolvida uma componente complementar na forma de um programa interativo, conforme previsto nos objetivos delineados para esta etapa.

2. Desenvolvimento

Para o projeto final, conseguimos implementar os modos descritos no guião (batch e interativo), bem como a totalidade das queries. Adicionalmente, o programa de testes foi revisto e aperfeiçoado, permitindo o cálculo de métricas de desempenho. O desenvolvimento manteve a qualidade de código já seguida na 1ª fase, com encapsulamento e modularidade aprimorados e documentação adicional.

2.1. Ajustes realizados

Após a disponibilização do *large dataset* pela equipa docente, procedeu-se à introdução de um conjunto de otimizações com o objetivo de melhorar o desempenho da aplicação. Estas otimizações incidiram, sobretudo, na eficiência ao nível do consumo de memória e do tempo de execução.

Em particular, alguns tipos de dados utilizados nas estruturas internas foram substituídos por tipos equivalentes mais compactos, adequados a valores de menor dimensão, como a utilização de *short* em vez de *int*, permitindo reduzir a memória. Adicionalmente, foram implementadas estruturas de dados nomeadamente matrizes com o objetivo de otimizar o tempo de execução das queries, permitindo cálculos diretos através de acessos indexados e evitando percursos repetidos sobre grandes volumes de dados.

2.1.1. Indicações dos Docentes

Durante a defesa da 1.ª fase, os docentes sugeriram diversos métodos relacionados com o encapsulamento e a modularidade, tendo-se, a seguir, procedido à implementação das recomendações, que se detalham de seguida.

Em relação ao encapsulamento, adotou-se a definição de estruturas opacas, confinando a implementação integral das mesmas aos ficheiros de código-fonte (.c) e restringindo as suas declarações aos ficheiros de cabeçalho (.h). Esta abordagem limita o acesso direto aos campos das estruturas, o qual é efetuado exclusivamente através de funções específicas. Para reforçar esta metodologia, implementaram-se *getters* nos respetivos módulos, assegurando uma camada adicional de proteção e controlo sobre a manipulação dos dados internos.

Relativamente à modularidade, a arquitetura do projeto foi reestruturada, tendo em conta diversos aspetos salientados na defesa da primeira fase, com o objetivo de assegurar uma clara separação de responsabilidades. Ao nível dos *Managers*, adotou-se uma distinção entre a definição das entidades singulares e as estruturas de dados que as armazenam. Esta separação é exemplificada no módulo dos passageiros, onde se distingue a manipulação de um passageiro individual (*entity*) da gestão da coleção de passageiros (*structure*). Paralelamente, foi desenvolvido um módulo de armazenamento genérico (*storage_generic*), que encapsula a lógica de alocação dinâmica e indexação, recorrendo

a *Hash-tables* e outras estruturas de armazenamento, sendo reutilizável nas diversas entidades, tais como aeronaves, companhias aéreas e voos. Os *Parsers* refletem, igualmente, esta lógica. Em vez de funções especializadas para cada entidade, implementámos funções genéricas de leitura e *parsing*, capazes de interpretar diferentes tipos de registos de forma unificada.

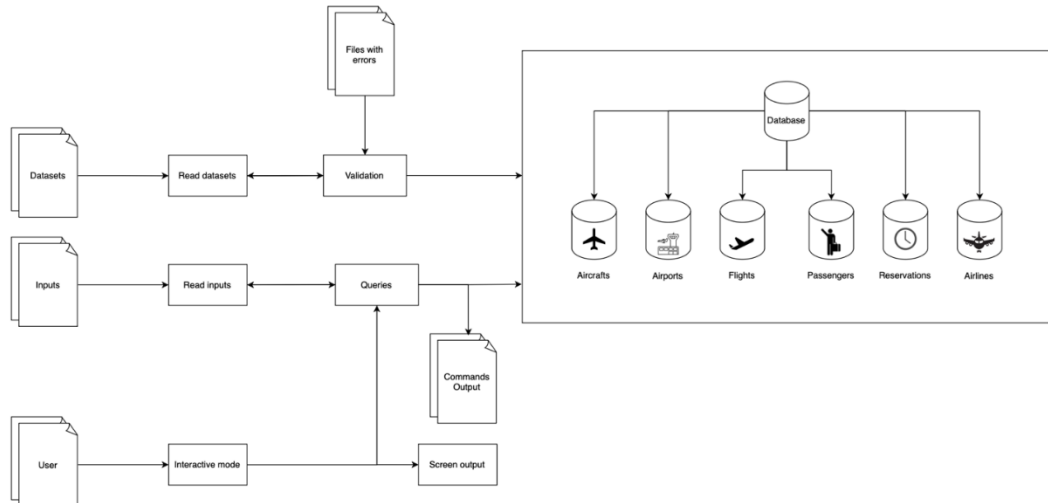
2.2. Arquitetura Aplicada

A arquitetura do sistema manteve a sua estrutura modular hierárquica, dividida em três camadas lógicas distintas, garantindo o princípio da separação de responsabilidades. As alterações efetuadas não incidiram sobre a arquitetura em si, mas sim sobre a reorganização de alguns ficheiros e a renomeação de módulos, com o objetivo de melhorar a clareza e a manutenção do código.

Managers: Esta camada constitui o núcleo de armazenamento em memória do sistema. Encontra-se segmentada por entidades principais (Aeroportos, Voos, Passageiros, Aeronaves e Reservas), onde cada módulo é subdividido em dois componentes fundamentais: a *Entity*, responsável pela definição do modelo de dados e encapsulamento dos atributos individuais; e a *Structure*, responsável pela gestão da coleção de dados e pela abstração das estruturas de armazenamento subjacentes (como *Hash-table* ou *Arrays*). Esta organização permite que as restantes camadas invoquem operações (por exemplo, *get_passenger_by_id*) sem dependência da implementação concreta dos dados.

Parsers: Esta camada assume a responsabilidade pela interface com fontes de dados externas. As suas principais funções incluem a leitura e *parsing* dos ficheiros CSV e a validação sintática e lógica dos registos. Adicionalmente, à medida que o *parsing* dos ficheiros é efetuado, esta camada procede à preparação e povoamento da base de dados em memória, garantindo que apenas dados corretos são inseridos nos *Managers* e ficando o sistema apto para a execução eficiente das queries.

Queries: Esta camada é responsável pela execução das queries, articulando e combinando dados provenientes dos diversos *Managers*. Compete-lhe aplicar filtros, realizar agregações e cálculos estatísticos. A sua implementação visa a eficiência, recorrendo frequentemente a dados pré-processados para garantir o cumprimento dos requisitos temporais de execução.



2.3. Queries

Com o objetivo de avaliar o desempenho do código desenvolvido, foi efetuada a análise do tempo médio de execução de cada query em diferentes máquinas.

Para tal, os testes foram realizados em dois Lenovo IdeaPad (configurações 1 e 2) e num MacBook Pro (configuração 3).

	Config. 1	Config. 2	Config. 3
CPU	AMD Ryzen 7 5700U	Intel Core i7-13620H	M3 Pro
RAM	16GB	16GB	18GB

Os dados apresentados sobre cada *query* correspondem a uma média de 3 execuções consecutivas numa dada configuração. Não foram apresentados os valores do uso de memória, uma vez que são consistentemente iguais e independentes do input e da configuração.

2.3.1. Query 1

Tem como objetivo apresentar o resumo operacional de um aeroporto, identificado pelo seu código IATA, incluindo o código, o nome, a cidade, o país, o tipo de aeroporto, bem como o número de passageiros que aterraram e o número de passageiros que partiram do aeroporto. Esta informação é gerida através de uma HashTable associada à entidade Aeroportos, permitindo um acesso direto e eficiente aos dados. Para garantir uma resposta com complexidade temporal constante, os totais de passageiros em voos de chegada e de partida, excluindo os voos com estado “Cancelled”, não são calculados no momento da consulta. Em alternativa, estes valores são mantidos como contadores persistentes na estrutura de cada aeroporto e são atualizados de forma incremental durante o processamento inicial do catálogo de voos. A seguir encontram-se os tempos de execução da *query*, repetida com 50 inputs diferentes:

<i>Dataset com erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.002	0.001	0.005
Tempo Total (ms)	0.123	0.052	0.268

2.3.2. Query 2

Tem como objetivo apresentar o Top N aeronaves com maior número de voos, por ordem decrescente. Em situações de empate no número total de voos, a ordenação é efetuada por ordem alfabética crescente do identificador da aeronave. Esta interrogação recebe como argumento o número de aeronaves a incluir no resultado e pode ainda admitir um filtro opcional por fabricante. Para a concretização deste objetivo, recorre-se a um array de aeronaves ordenado através do algoritmo QuickSort, tendo como critério principal o número total de voos, em ordem decrescente, e como critério de desempate o identificador alfanumérico da aeronave, em ordem crescente. A seguir encontram-se os tempos de execução da query:

<i>Dataset com erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.027	0.022	0.008
Tempo Total (ms)	1.326	1.082	0.415

2.3.3. Query 3

Tem como objetivo identificar o aeroporto com mais partidas entre duas datas fornecidas como argumento (data inicial e data final), devolvendo como resultado o aeroporto com o maior número de partidas e o respetivo total. Com o objetivo de melhorar o tempo de execução da query, otimizamos o processamento através da utilização de uma matriz bidimensional de números inteiros que armazena valores acumulados de partidas por data e por aeroporto. Nesta estrutura, cada linha corresponde a uma data, ordenada cronologicamente, e cada coluna corresponde a um aeroporto, identificado pelo seu código IATA. Na query, a função percorre todas as colunas associadas aos aeroportos e calcula o número total de partidas entre a data final e inicial (faz apenas a diferença entre as duas linhas), evitando a necessidade de percorrer as datas intermédias. O aeroporto cujo valor acumulado de partidas seja máximo é então identificado como resultado da query. Em situações de empate, aplica-se como critério de desempate a ordem lexicográfica do código IATA. Tendo isto em conta, o tempo de execução desta query encontra-se a seguir:

<i>Dataset com erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.009	0.014	0.005
Tempo Total (ms)	0.445	0.679	0.229

2.3.4. Query 4

Tem como objetivo identificar o passageiro que esteve mais vezes presente no top 10 de despesas semanais durante um período específico. Durante a fase de preparação, os registos financeiros das reservas são agregados semanalmente, calculando-se o montante total despendido por cada passageiro. Subsequentemente, são isolados os dez passageiros com maior despesa acumulada em cada semana, sendo os respetivos identificadores armazenados num *array* de dimensão fixa associado à semana em questão. Para determinar o passageiro predominante num dado intervalo de tempo, o algoritmo converte as datas fornecidas nos índices das semanas correspondentes e percorre exclusivamente os vetores dos top 10 dessas semanas. Recorre-se a uma *Hash-table* temporária para contabilizar o número de ocorrências de cada passageiro neste subconjunto restrito de dados, identificando-se o máximo. Em caso de empate na frequência, aplica-se o critério de desempate pelo menor identificador numérico.

<i>Dataset</i> com erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.011	0.014	0.007
Tempo Total (ms)	0.556	0.718	0.325

2.3.5. Query 5

Tem como objetivo identificar as N companhias aéreas com a maior média de atraso. Para o efeito, utiliza-se um *array* de companhias, no qual são acumuladas estatísticas exclusivamente relativas aos voos com estado “*Delayed*”. O atraso é quantificado pela diferença temporal, em minutos, entre a hora efetiva de partida e a hora prevista, permitindo calcular a média aritmética, através do quociente entre o atraso total e o número de voos atrasados. O *array* é previamente ordenado de forma decrescente com base na média dos atrasos, pelo que a query se limita a selecionar os N primeiros elementos. A seguir encontram-se os tempos de execução da *query*, repetida com 50 inputs diferentes:

<i>Dataset</i> com erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.008	0.007	0.004
Tempo Total (ms)	0.402	0.356	0.205

2.3.6. Query 6

Tem como objetivo identificar o aeroporto de destino predominante para uma determinada nacionalidade. Para isso utilizou-se uma matriz pré-calculada. Esta matriz organiza as nacionalidades em linhas e os aeroportos em colunas, com cada célula contendo um inteiro que regista o número de passageiros de uma dada nacionalidade cujo destino corresponde ao aeroporto representado na coluna correspondente. Durante o carregamento inicial do catálogo de passageiros e voos, cada nacionalidade é atribuída a um identificador numérico que permite o acesso à linha pertinente. Sempre que um

passageiro viaja para um determinado aeroporto, o contador da célula correspondente é incrementado. Para determinar o aeroporto predominante, acede-se diretamente à linha da nacionalidade e percorrem-se as colunas dessa linha para identificar o valor máximo. Em caso de empate, aplica-se um critério de desempate predefinido, como a ordem lexicográfica do código IATA.

<i>Dataset com erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.009	0.006	0.004
Tempo Total (ms)	0.431	0.315	0.213

2.4. Programa Interativo

O modo interativo, um requisito da segunda fase, disponibiliza uma interface gráfica no terminal que permite ao utilizador indicar o caminho dos ficheiros de dados e submeter queries de forma interativa. Para a sua implementação recorreu-se à biblioteca *ncurses*, escolhida por simplificar a criação de elementos gráficos no terminal. A interface solicita ao utilizador que escolha uma das seis queries disponíveis e introduza os argumentos correspondentes. Caso os parâmetros fornecidos sejam inválidos, o programa apresenta uma mensagem de erro informando a situação; se forem válidos, a *query* é executada e os resultados são apresentados ao utilizador.

A arquitetura do modo interativo segue princípios de modularidade e separação de responsabilidades: a camada de apresentação foi organizada em componentes reutilizáveis (por exemplo, modais e menus), isolando-se a lógica de interface da lógica de processamento das *queries* e do acesso aos dados.

2.5. Dificuldades sentidas

O desenvolvimento desta fase apresentou vários desafios, sendo a implementação da Query 4 o mais significativo. Uma das primeiras dificuldades consistiu na definição de um modelo para representar as semanas de forma consistente, que permitisse o mapeamento direto entre datas e semanas. Para esse efeito, optou-se pela organização das semanas num array indexado, possibilitando o acesso direto. Outra dificuldade foi a criação de estruturas de dados adequadas para suportar os cálculos necessários. Em particular, tornou-se necessário desenvolver uma estrutura intermédia que armazenasse, para cada semana, o valor total gasto por cada passageiro e uma segunda estrutura específica para armazenar, para cada semana, apenas o Top 10 de passageiros por despesa. Por fim, a articulação entre estas estruturas pré-processadas e a lógica da query constituiu outro desafio importante.

Além das questões específicas da Query 4, o desenvolvimento do código como um todo apresentou desafios relacionados com a modularidade e o encapsulamento. Foi necessário organizar as funções e as estruturas de dados, evitando dependências excessivas entre módulos.

3. Conclusão

Concluindo, o desenvolvimento da 2ª Fase foi realizado da melhor forma possível, devido sobretudo à sólida base obtida já 1ª Fase, mas também devido à implementação dos conselhos e correções dos docentes. Desta forma, atingimos todos os objetivos estabelecidos para o projeto final de Laboratórios de Informática III.